Lecture 14: Python Modules (Chapter 7)

교재 Ch. 7.1, 7.2, 7.3

유준수 (Joon Soo Yoo, Ph.D.)

2026년 4월 2일

방산첨단보안연구실 (DAS Lab)
전북대학교 첨단방위산업학과

7.1 Module Basics

코드의 부품화: 모듈 정의와 임포트

모듈(Module)이란 무엇인가?

파이썬에서 모듈은 함수 정의나 문장들을 담고 있는 **.py 파일** 그 자체를 의미합니다.

- **파일명이 곧 모듈명:** `greetings.py` 파일은 `greetings`라는 이름의 모듈이 됩니다.
- **목적:** 코드가 길어지고 복잡해질 때, 기능별로 파일을 나누어 관리(모듈화)하기 위함입니다.

모듈 예시 (greetings.py)

```
"""표준 인사를 출력하는 함수 모음"""
def hello(): print("Hello!")
def bye(): print("Goodbye!")
```

Modular Design 팁

모듈은 보통 실행을 목적으로 하기보다, 다른 프로그램에서 **부품처럼 호출**되기 위해 존재합니다.

모듈 정의하기 (area.py)

여러 가지 도형의 넓이를 계산하는 함수들을 하나의 파일에 모아보겠습니다.

area.py 설계

```
import math


def square(side): return side**2
def circle(radius): return math.pi * radius**2
def cube(side): return 6 * side**2
```

- **특징:** 위 코드를 실행해도 아무런 결과가 출력되지 않습니다. 함수가 **정의**만 되었을 뿐, **호출**되지 않았기 때문입니다.
- **Docstring:** 함수 첫 줄에 `"""` 설명 `"""`을 적어 모듈의 사용법을 명시하는 것이 표준입니다.

모듈 불러오기 (Importing)

정의된 모듈을 사용할 때는 `import` 키워드를 사용하며, 확장자(`.py`)는 붙이지 않습니다.

area 모듈 사용 예시

```
import area   # area.py를 불러옴


print(area.square(10))   # 결과: 100
print(area.circle(5))   # 점(.)을 찍어 모듈 내부 함수에 접근
```

도트 표기법 (Dot Notation)

`area.circle(5)`와 같이 모듈명.함수명() 형태로 호출합니다. 이는 다른 모듈에 같은 이름의 함수가 있더라도 충돌을 방지해주는 안전 장치입니다.

예시: 단위 변환 모듈 (conversion.py)

Mission: 아래 기능들을 담은 `conversion.py` 모듈을 만드세요.

1. `cel2fah(c)`: 섭씨 $\rightarrow$ 화씨 $(9/5 \times c + 32)$
2. `km2mi(km)`: 킬로미터 $\rightarrow$ 마일 $(km/1.60934)$

사용 예시 (Main Program)

```
import conversion
print(conversion.km2mi(180)) → 111.84...
```

7.2 Importing names

특정 함수만 골라오기: `from` 키워드와 이름 충돌

from 키워드: 특정 기능만 직접 호출하기

모듈 전체가 아닌, 필요한 함수만 콕 집어서 가져올 수 있습니다.

- **사용법:** `from 모듈명 import 함수1, 함수2`
- **장점:** 모듈 이름을 앞에 붙이지 않고 함수 이름만으로 즉시 호출 가능합니다.

from 사용 예시 (area 모듈)

```
from area import triangle, cylinder
print(triangle(1, 2))   # area.triangle() 이라고 안 해도 됨!
print(cylinder(3, 4))
```

주의: NameError

임포트하지 않은 함수를 호출하면 파이썬은 해당 이름을 찾지 못해 `NameError`를 발생시킵니다.

이름 충돌 (Name Collisions)

서로 다른 곳에서 온 함수들이 **같은 이름**을 가질 때 발생하는 문제입니다.

충돌 시나리오: "가장 늦게 정의된 것이 이긴다"

```
from area import cube


def cube(x):
    return x ** 3   # 임포트한 cube를 덮어씀 (Name Collision)


print(cube(2))   # 결과: 8 (area.cube가 아닌 로컬 함수 호출)
```

- **특징:** 파이썬은 이름 충돌 시 에러를 내지 않고 **가장 최근의 정의**를 사용합니다.

충돌 방지 전략: 편리함 vs 안전성

프로그래머는 상황에 따라 어떤 임포트 방식을 쓸지 결정해야 합니다.

Import 방식 비교

방식	장점	단점 (리스크)
`import area`	`area.cube()` 처럼 출처가 명확하여 **충돌 방지**에 유리	코드가 길어짐
`from area import ...`	코드가 간결하고 메모리 효율적	기존 함수나 변수 이름을 **덮어쓸 위험**이 큼

팁: Explicit is better than implicit

복잡한 알고리즘을 구현할 때는 가급적 `import module` 방식을 사용하여, 해당 함수의 출처를 명확히 드러내는 것이 유지보수에 유리합니다.

예시 2: 무엇이 문제일까요? (Find the Bug)

문제 상황: 이름 충돌 (Name Collision)

```
from area import sphere   # 구의 넓이(Surface Area) 함수
from volume import sphere   # 구의 부피(Volume) 함수

print("Ball Area:", sphere(10))   # 과연 넓이가 출력될까요?
```

질문

위 코드를 실행했을 때 예상과 다른 결과가 나오는 이유는 무엇일까요?

7.3 Top-level code

부수 효과(Side Effects)와 `__name__`의 비밀

부수 효과 (Side Effects)란?

모듈을 `import`할 때, 파이썬은 해당 파일의 코드를 **위에서 아래로 모두 실행**합니다.

- **문제점:** 함수 정의 외에 `print()`나 `input()` 같은 코드가 파일 최상단(Top-level)에 있으면, 임포트하는 순간 의도치 않게 실행되어 버립니다.

위험한 모듈 예시 (volume.py)

```
import math

def sphere(r): return 4/3 * math.pi * r**3

print(sphere(10))   # <- 부수 효과 발생!
```

보안 경고

다른 프로그램에서 `import volume`을 하는 순간, 원치 않는 계산 결과가 화면에 출력됩니다. 만약 저 위치에 데이터 삭제 명령이 있다면 시스템에 치명적일 수 있습니다.

__name__ : 파이썬의 마법 변수

파이썬은 모든 모듈에 __name__이라는 특수 변수를 자동으로 할당합니다.

- **직접 실행 시:** 해당 파일의 __name__은 "__main__"이 됩니다.
- **임포트될 시:** 해당 파일의 __name__은 **모듈 이름**이 됩니다.

__name__ 값의 변화

실행 방식	__name__ 변수의 값
python test.py (직접 실행)	"__main__"
import test (다른 곳에서 호출)	"test"

표준 관습: `if __name__ == "__main__":`

모듈이 라이브러리로 쓰일 때는 실행되지 않고, **테스트나 직접 실행 시에만** 작동하게 하는 안전 장치입니다.

안전한 모듈 설계 (area.py)

```
def circle(r): return 3.14 * r**2


if __name__ == "__main__":
    print("모듈 직접 실행 테스트 중...")
    print(circle(10))   # 임포트 시에는 실행되지 않음!
```

- **Dunder(던더):** 밑줄 두 개(__)로 시작하고 끝나는 변수를 'Double Underscore'의 약자로 **Dunder** 또는 마법 변수라고 부릅니다.

예시 3: 모듈 테스트 코드 격리

Mission: `conversion.py`에 테스트 기능을 추가하세요.

1. 기존 단위 변환 함수들 아래에 `if __name__ == "__main__":` 구문을 추가합니다.
2. 해당 구문 안에서 `cel2fah(0)`, `km2mi(1)` 등을 테스트하는 `print` 문을 작성하세요.
3. 다른 파일에서 이 모듈을 임포트했을 때 테스트 결과가 나오지 않는지 확인하세요.

Any Questions?